AL-QIMMAH BOOTCAMP · 2026

QuoteIt

Software Requirements Specification (SRS)

An interactive web platform for publishing, sharing, and managing quotes — with a database-backed quote feed and a like/unlike feature.

Document version: 2.0 (final, updated)

Project type: Individual project

Track: Programming & Development

Date: 27 October 2026

Implementation: Laravel 13 · PHP 8.5 · Blade · Tailwind CSS · SQLite

---

This document updates the provided SRS to reflect the final implemented system, including the new Like feature.

Table of Contents

1. Introduction
    1.1 Purpose
    1.2 Scope
    1.3 Definitions
    1.4 References
2. Overall Description
    2.1 Product Perspective
    2.2 User Classes
    2.3 Operating Environment
    2.4 Constraints and Assumptions
3. System Features
    3.1 View Quotes
    3.2 Create Account
    3.3 Log In
    3.4 Log Out
    3.5 Share a Quote
    3.6 Edit Own Quote
    3.7 Delete Own Quote
    3.8 Like / Unlike a Quote (New Feature)
4. External Interface Requirements
5. Non-Functional Requirements
6. Data Requirements
7. Requirements Traceability
8. Changes from the Original SRS
9. Future Enhancements
10. Appendix: Setup and Demo Account

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of **QuoteIt**, a web platform for publishing and managing quotes. It is the reference document for the final version of the project and describes what the system does, how users interact with it, and the constraints it must satisfy.

1.2 Scope

QuoteIt allows visitors to browse all published quotes without an account. Registered users can create an account, log in, publish quotes, edit or delete their own quotes, and log out. The final version adds one new feature: **like/unlike with persistent like counts**. All data is stored in a relational database; no quote data is hardcoded in the views.

The system deliberately excludes features that are not required by the project brief, such as private messaging, quote categories, password reset, and moderation tools.

1.3 Definitions

Term	Definition
Guest	An unauthenticated visitor who can browse quotes.
Registered User	An authenticated account holder who can publish and manage their own quotes.
Quote	A short text of 3–155 characters published by a registered user.
Owner	The user who created a quote; only the owner may edit or delete it.
Like	A persistent association between one user and one quote. A user may like a quote at most once.
Like count	The number of distinct users who currently like a quote.
CSRF	Cross-Site Request Forgery protection token included in every state-changing form.

1.4 References

- Al-Qimmah Bootcamp — Final Project Guide, 2026.
- Provided QuoteIt frontend starter code.
- Laravel 13 documentation: routing, validation, Eloquent, authentication, Blade.

2. Overall Description

2.1 Product Perspective

QuoteIt is a standalone Laravel web application following the MVC pattern. Blade templates provide the presentation layer, controllers handle requests and validation, Eloquent models map the domain entities, and

migrations/seeders create and populate the database. The public quote feed is the entry point of the application.

2.2 User Classes

User class	Capabilities
Guest	Browse all quotes (text, author name, publication date, like count). Open sign-up and login pages.
Registered User	Everything a guest can do, plus publish quotes, edit/delete own quotes, like/unlike any quote, and log out.

2.3 Operating Environment

Component	Technology
Server language	PHP 8.5
Framework	Laravel 13
Database	SQLite (development); schema is portable to MySQL
View layer	Blade templates with template inheritance and components
Styling	Tailwind CSS compiled with Vite
Client	Any modern web browser; responsive on mobile and desktop

2.4 Constraints and Assumptions

- All views must use Blade template inheritance (`@extends`, `@yield`) and Blade components.
- Every form field must display a short hint describing the allowed input.
- Passwords must be stored hashed; plain-text passwords must never be persisted.
- Only the owner of a quote may edit or delete it.
- The quotes page must load within 3 seconds.
- Users are assumed to have a valid email address and a modern browser.

3. System Features

> **FR-01 — View quotes.** The system shall display every published quote to every visitor, including guests, showing the quote text, the author's name, and the publication date.

3.1 View Quotes

Actors: Guest, Registered User. **Preconditions:** none.

Main flow: The user opens the home page → the system retrieves all quotes with their authors and like counts from the database → the system renders the quotes list ordered from newest to oldest → each card shows the quote text, author name, date, and like count.

Postconditions: The quotes list is visible; guests see login/sign-up links; registered users additionally see the share form and like buttons.

Acceptance criteria: A guest visiting `/` sees the quotes with author and date; the data comes from the database, not from hardcoded view data.

> **FR-02 — Create account.** The system shall allow a visitor to register with a name of 2–50 characters, a valid unique email address, and a password of at least 8 characters.

3.2 Create Account

Actor: Guest. **Preconditions:** the visitor is not authenticated.

> **Main flow:** The visitor opens the sign-up page → enters full name, email, password, and password confirmation → submits the form → the system validates all fields → the password is hashed → the user record is created → the user is logged in and redirected to the home page.

Alternative flows:

- Name shorter than 2 or longer than 50 characters → validation error displayed next to the field.
- Email is invalid or already registered → validation error displayed next to the field.
- Password shorter than 8 characters or confirmation does not match → validation error displayed next to the field.

Postconditions: A new user exists with a hashed password, and an authenticated session is active.

Acceptance criteria: Valid data creates an account and logs the user in; invalid data is rejected with a visible message and the form is not submitted to the database.

> **FR-03 — Log in.** The system shall authenticate a registered user using their email address and password.

3.3 Log In

Actor: Guest (registered user). **Preconditions:** an account exists.

> **Main flow:** The user opens the login page → enters email and password → the system verifies the credentials against the stored hash → the session is regenerated → the user is redirected to the home page.

Alternative flow: Credentials do not match → a general error is shown and the email is preserved in the form.

Acceptance criteria: Correct credentials authenticate the user; incorrect credentials do not create a session.

> **FR-04 — Log out.** The system shall allow an authenticated user to end their session.

3.4 Log Out

Actor: Registered User. **Preconditions:** the user is authenticated.

Main flow: The user presses “Log out” in the header → the system invalidates the session and regenerates the CSRF token → the user is redirected to the home page as a guest.

Acceptance criteria: After logout, protected actions are no longer available and the header shows login/sign-up links.

> **FR-05 — Share a quote.** The system shall allow an authenticated user to publish a quote of 3–155 characters.

3.5 Share a Quote

Actor: Registered User. **Preconditions:** the user is authenticated.

Main flow: The user writes the quote in the share form → submits → the system validates the length → the quote is stored with the authenticated user's ID and the current timestamp → the page reloads and the new quote appears at the top of the list.

Alternative flows: Fewer than 3 or more than 155 characters → a validation error is displayed and nothing is stored.

Postconditions: The quote is persisted in the database and visible to all visitors.

> **FR-06 — Edit own quote.** The system shall allow a user to edit only the quotes they created.

3.6 Edit Own Quote

Actor: Registered User (owner). **Preconditions:** the user is authenticated and owns the quote.

Main flow: The owner selects “Edit” on their quote → the edit form opens with the current text → the owner changes the text and submits → the system verifies ownership and validates the length → the quote is updated.

Alternative flows: The user is not the owner → the system responds with HTTP 403 and does not reveal the edit form. Invalid length → validation error.

Acceptance criteria: Owners can update their quotes; other users cannot access the edit route or persist changes.

> **FR-07 — Delete own quote.** The system shall allow a user to delete only the quotes they created.

3.7 Delete Own Quote

Actor: Registered User (owner). **Preconditions:** the user is authenticated and owns the quote.

Main flow: The owner selects “Delete” and confirms → the system verifies ownership → the quote and its likes are deleted → the list refreshes without the quote.

Alternative flow: A non-owner attempts deletion → HTTP 403; the quote remains unchanged.

> **FR-08 — Like / unlike a quote (new feature).** The system shall allow an authenticated user to like or unlike any quote and shall display the stored like count on every quote.

3.8 Like / Unlike a Quote (New Feature)

Actor: Registered User. **Preconditions:** the user is authenticated and the quote exists.

> **Main flow:** The user presses the heart button on a quote → the system toggles the like for that user → the association is stored in `quote_likes` → the page reloads and the heart state and count update.

Alternative flow: A guest sees the like count but not an interactive like button; guests are invited to log in.

Business rules:

- A user can like a given quote at most once; pressing again removes the like.
- Like counts are computed from the database and are visible to guests and registered users.
- Deleting a quote also deletes its likes (cascade).

Acceptance criteria: Liking increments the count, unliking decrements it, the count persists after a reload, and the toggle only works for authenticated users.

4. External Interface Requirements

4.1 User Interfaces

Page	Route	Purpose
Quotes list	`GET /`	Public feed of quotes, share form for authenticated users, like buttons, owner controls.
Sign up	`GET /signup`	Account creation form with field hints and inline errors.
Log in	`GET /login`	Email/password login form with field hints and errors.
Edit quote	`GET /quotes/{id}/edit`	Owner-only edit form.

The interface is responsive, uses a dark cybersecurity visual theme for quote cards, and uses Blade components for inputs, text areas, hints, error messages, alerts, and branding.

4.2 Form Fields and Hints

Form	Field	Rule	Hint shown
Sign up	Full name	Required, 2–50 characters	“2-50 characters.”
Sign up	Email	Required, valid, unique	“Use a valid email address.”
Sign up	Password	Required, minimum 8 characters	“At least 8 characters.”

Sign up	Confirm password	Must match the password	"Repeat the password exactly."
Log in	Email	Required, valid email	"Use the email you signed up with."
Log in	Password	Required	"At least 8 characters."
Log in	Remember	Optional checkbox	"Keep me signed in on this device."
Share / Edit	Quote text	Required, 3–155 characters	"3-155 characters."

4.3 Software Interfaces

- **Database:** SQLite via Laravel's PDO driver; Eloquent models `User` and `Quote`.
- **Session storage:** database-backed sessions.
- **Asset pipeline:** Vite builds the Tailwind CSS and JavaScript assets.

5. Non-Functional Requirements

Category	Requirement	Implementation
Security — password storage	Passwords must be hashed, never stored as plain text.	Laravel `hashed` cast on `User::$password`; bcrypt hashing.
Security — authorization	Only the owner may edit or delete a quote; others may only read it.	Authenticated routes plus server-side ownership check returning HTTP 403.
Security — request forgery	State-changing forms must be protected.	CSRF tokens on every POST/PUT/DELETE form; session regenerated on login and logout.
Usability — hints	Every form field must show a short hint about allowed input.	Reusable `<x-form-hint>` component on every field.
Usability — first quote	A new user must be able to publish a first quote in under one minute.	Share form is at the top of the home page after login; a single field and one submit button.
Performance	The quotes page must load within 3 seconds.	Single indexed query with eager loading and aggregate like count; compiled assets; measured locally at well under 1 second.
Reliability / integrity	Data must remain consistent.	Foreign keys with cascade delete; unique constraint on `(quote_id, user_id)`.

Maintainability	The code must follow the framework conventions.	MVC separation, migrations, seeders, reusable Blade components, automated tests.
Portability	The application should run on a standard PHP/Laravel environment.	Standard Laravel structure; environment-driven database configuration.

6. Data Requirements

6.1 Entities

users

Column	Type	Constraints
id	BIGINT UNSIGNED	Primary key, auto-increment
name	VARCHAR(255)	Required; validated to 2–50 characters
email	VARCHAR(255)	Required, unique
email_verified_at	TIMESTAMP	Nullable
password	VARCHAR(255)	Required, hashed
remember_token	VARCHAR(100)	Nullable
created_at / updated_at	TIMESTAMP	Nullable

quotes

Column	Type	Constraints
id	BIGINT UNSIGNED	Primary key, auto-increment
user_id	BIGINT UNSIGNED	Foreign key → `users.id`, required, cascade delete
content	VARCHAR(155)	Required; validated to 3–155 characters
created_at / updated_at	TIMESTAMP	Nullable

quote_likes

Column	Type	Constraints
id	BIGINT UNSIGNED	Primary key, auto-increment
quote_id	BIGINT UNSIGNED	Foreign key → `quotes.id`, required, cascade delete
user_id	BIGINT UNSIGNED	Foreign key → `users.id`, required, cascade delete

created_at / updated_at	TIMESTAMP	Nullable
Unique constraint: (`quote_id` , `user_id`) — one like per user per quote.		

6.2 Relationships

- A **user** has many **quotes**; each quote belongs to exactly one user.
- A **user** has many **likes**; each like belongs to one user.
- A **quote** has many **likes**; each like belongs to one quote.
- Users and quotes form a many-to-many relationship through `quote_likes` .

The complete entity relationship diagram is provided separately as **QuoteIt_ERD.pdf**.

7. Requirements Traceability

Req.	Requirement	Implementation	Verification
FR-01	Show all quotes with text, author, date	`QuoteController@index` ; `welcome.blade.php` quote cards	Guest HTTP check; database-backed list verified
FR-02	Sign up with name 2–50, email, password ≥ 8	`AuthController@signup` validation; sign-up Blade form	Invalid and valid input tested
FR-03	Log in with email and password	`AuthController@login` ; Laravel session guard	Correct and incorrect credentials tested
FR-04	Log out	`AuthController@logout` ; header form	Session invalidated tested
FR-05	Share quote 3–155 characters	`QuoteController@store` validation and persistence	Boundary validation tested
FR-06	Edit only own quotes	`QuoteController@edit/update` with ownership check	Owner success, non-owner 403 tested
FR-07	Delete only own quotes	`QuoteController@destroy` with ownership check	Owner success, non-owner 403 tested
FR-08	Like / unlike with stored count	`LikeController@toggle` ; `quote_likes` table	Like, unlike, persistence tested
NFR-1	Passwords hashed	`User` model `hashed` cast	`Hash::check` verified
NFR-2	Owner-only modification	Server-side 403 ownership checks	Cross-user attempts tested
NFR-3	Hints on every field	`x-form-hint` on all fields	Page inspection
NFR-4	Quotes page under 3 seconds	Eager loading, aggregate count, compiled assets	Local load timing
NFR-5	Real database integration	Migrations, seeders, Eloquent models	Seeded data rendered from SQLite

8. Changes from the Original SRS

- **New feature — Likes:** added the `quote_likes` table, toggle action, and like counts. This satisfies the mandatory "at least one new feature" requirement.
- **Database integration:** quotes, users, and likes are persisted in SQLite instead of static view data.
- **Authentication:** completed sign-up, login, logout, validation, and session handling.
- **Authorization:** added owner-only edit/delete with server-side enforcement.
- **User interface:** restored the quote feed with Blade components, responsive cards, hints, error messages, and success alerts.
- **Seed data:** added a demo account and eight cybersecurity quotes with varied authors and dates.
- **Testing:** enabled database refresh for feature tests and verified the full flow over HTTP.

9. Future Enhancements

- Bookmarks/favorites for saving quotes.
- Search and filter by author, date, or keyword.
- Quote categories and tags.
- Profile pages with a user's quotes and liked quotes.
- Pagination or infinite scrolling for large quote collections.
- Password reset and email verification.

10. Appendix: Setup and Demo Account

10.1 Demo Account

Field	Value
Email	`demo@quoteit.com`
Password	`password123`

10.2 Run the Project

1. Install PHP 8.3+ and Composer.
2. Run `composer install`.
3. Copy `.env.example` to `.env` and run `php artisan key:generate`.
4. Ensure `database/database.sqlite` exists, then run `php artisan migrate --seed`.
5. Run `npm install && npm run build` (or `npm run dev`).
6. Run `php artisan serve` and open `http://127.0.0.1:8000`.